Daak and Arkiv: the tracks we enter, and what is built

Arkiv is Daak's only database: people, bookings, handoffs and daily orders, each with the lifetime it should have

In 60 seconds

1. There is no Postgres and no server disk. Every piece of state lives as a typed Arkiv entity written by one publisher wallet.
2. Expiry is the design: a handoff record is needed for the day the parcel travels, so it expires after a day; people and bookings after 30.
3. What users see reads Arkiv directly: the receiving shop's tracking page, the public deliveries list, the drivers page and the app.

Tracks

Build on Arkiv, from scratch
`$3,000`

Swarm bonus
`$500 (its own brief)`

Daak enters three partners, the event's limit: Hedera, 0G and Arkiv (with Swarm).

What the track asks, and where Daak stands

Writes with @arkiv-network/sdk	SDK 1.0.1 on Rabdi (chain 7794095), typed schemas	Live
At least one feature users see reads from Arkiv	/track/<code>, /live, /drivers and the app all read it	Live
Wallet addresses and a runnable query in the README	Owner 0x610F...46fe; npx tsx scripts/arkiv-query.mts needs no keys	Live
/arkiv/schema.md	Five schemas with indexed columns, lifetimes, writers and readers	Live
One file per criterion: friction, reach, why, how	friction.md, reach.md, why-arkiv.md, how-you-use-arkiv.md	Live

Every piece, where it lives, and its proof

daak.badge (people)	Name, role, bus and stretches in the clear; phone, UPI and ID details sealed; the phone's signing key and the ID decision patched in place `web/lib/arkiv.ts`
daak.consignment (bookings)	Route, km per leg, each driver's share, fee in paise, weight (a band, or kg:24|60x50x40); written before any money moves `web/app/api/book`

daak.handoff	The 0G verdict, HCS sequence, photo hash, GPS, the sealed Swarm reference and the 0G Storage root; expires after a day `web/app/api/handoff`	
daak.standing (daily orders)	Booked by the 04:00 IST run; the day's parcel tag patched in when the shop taps it `web/lib/standing.ts`	
daak.pref	A driver's Bachat setting; the payout reads the newest `web/lib/arkiv.ts`	
Queries	Newest booking for an NFC serial, every handoff of a code, drivers by stretch, paged lists `web/lib/parcels.ts`	Runnable query

Honest limits

- One publisher wallet writes everything; shops and bus operators reading is open, but writing their own records is the next step.
- A dispute window longer than a day would need a handoff's expiry extended when a claim is filed.
- Testnet (Rabdi); costs on mainnet are not measured yet.

To ask the mentor

- Can an entity's expiry be extended in place when a claim is filed, or should we write a new one?
- What is the expected index lag after a write? We retry for up to 10 s.
- Would you model the shop and the bus operator as their own writers, and how would owners share one entity type?
- Any plan for ordering by an unindexed column to error instead of returning an empty page?

In the repo: arkiv/schema.md, arkiv/friction.md, arkiv/reach.md, arkiv/why-arkiv.md, arkiv/how-you-use-arkiv.md